TARGET ARCHITECTURE

Compliance Data Store – Target Design

Data models and attributes with reasons, relationships, processing, the final data store and where each column’s value comes from, the API, and a build guide for the whole system

System	NestJS + Prisma + PostgreSQL, alongside the appointment data store. Power BI keeps the PW and MP dashboards
Code	github.com/shivshankarkannoujiya/compliance-datastore (rules, scoring and storage modules on main)
Based on	Folders 01–07 (seven recordings), 00_Overview and Compliance_Data_Store_Logic.pdf (DSL §n, findings DS-nn)
Report date	16 October 2026
Status keys	**Built**: in the repo with tests. **Planned**: designed here, not yet built

In one paragraph. Today eleven Excel tools prepare data and paste it into Compliance Data.xlsm, where formulas and Power Query score it for Power BI. The new system keeps one PostgreSQL database with real IDs, an append-only record of every completion, versioned settings, and a scoring step that runs once per publish and is approved by a second person. Each source system gets an importer instead of copy-paste. Power BI reads database views.

Contents

1. Design Principles 3

2. Data Model Overview 4

3. Tables and Attributes 5

3.1 subject . . . 5
3.2 subject_alias . . . 5
3.3 compliance_type . . . 5
3.4 compliance_type_version . . . 6
3.5 requirement . . . 6
3.6 compliance_record . . . 7
3.7 publish_run . . . 7
3.8 published_requirement_status . . . 8
3.9 published_type_score, published_sub_group_score, published_group_score . . . 8
3.10 policy_version (planned) . . . 9
3.11 spc_rule_version (planned) . . . 9
3.12 import_batch (planned) . . . 9
3.13 import_row (planned) . . . 10
3.14 source_case (planned, restricted) . . . 10
3.15 measurement (planned) . . . 10
3.16 audit_event (planned) . . . 11

4. Relationships 12

5. Processing 13

5.1 Pipeline . . . 13
5.2 Processing per area (from the recordings) . . . 13
5.3 Monthly cycle . . . 13
5.4 Rules that keep processing safe . . . 13

6. Final Data Store and Column Lineage 14

6.1 reporting_requirement_status (one row per requirement in the latest published run) . . . 14
6.2 reporting_type_score / reporting_sub_group_score / reporting_group_score . . . 14
6.3 Where compliance_record values come from, by source . . . 14

7. API 15

7.1 Conventions . . . 15
7.2 Endpoints . . . 15

8. Build Guide 16

8.1 Phases . . . 16
8.2 Non-functional requirements . . . 16
8.3 Testing strategy . . . 16
8.4 Risks and mitigations . . . 16
8.5 What I need from you and the client . . . 16

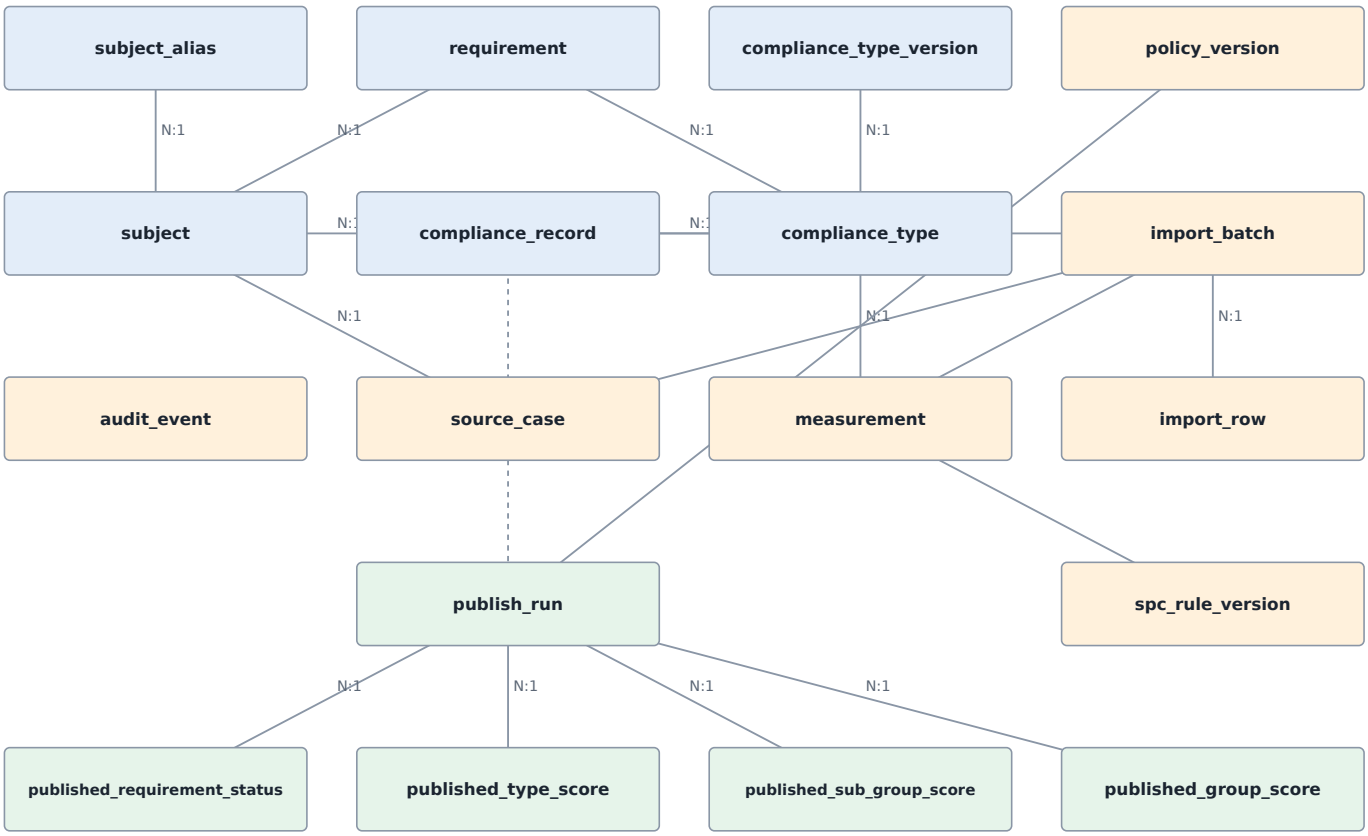
1. Design Principles

Every table and column below follows from these rules. Each answers a problem seen in the recordings.

Principle	What it means	Problem it solves (evidence)
IDs, not names	Every person, site, type and record has a UUID; names are aliases	Name text as the only key in every tool; renames and spelling variants break links (Overview §5)
Append-only evidence	Records are added, never edited or deleted; mistakes are voided with a reason and replaced	Overwrites (01), monthly delete-and-replace (04), no audit trail
Versioned settings	Catalogue, scoring policy and SPC parameters change by adding a version with a start date	Thresholds and core flags edited live (06), “only Matt knows”
Point-in-time	Every read takes an as-of date; past results can be reproduced	TODAY() in formulas changes results with the day the file is opened
One calculation, then publish	Scoring runs once into a draft; a second person approves; Power BI sees only published runs	Self-referencing query, “refresh twice”, Last Updated typed by hand (DS-09)
Rules as tested code	The rules engine is pure TypeScript with unit tests; open decisions are policy settings	Logic hidden in formulas; 17 rows marked Unclear (DS-17)
Importers, not paste	Each source has an adapter with a header contract, validation and duplicate checks	Copy-paste of Datix, SystmOne, Survicate, ESR exports
Restricted patient data	Patient-level fields live only in restricted tables, never in reporting views	NHS numbers and narratives in shared workbooks (02, 03, 05, 06)

2. Data Model Overview

Eighteen tables in four groups. Blue and green are built; amber is planned. Each table has one owning module that alone writes it.



Built (core) Built (publishing) Planned Dashed: run reads records point-in-time

Table	Group	Status	Owner	One row is…
subject	Core	Built	subjects	a person, site or the organisation
subject_alias	Core	Built	subjects	one spelling of a subject’s name
compliance_type	Core	Built	catalogue	a compliance type (DBS, Moderate Harm…)
compliance_type_version	Core	Built	catalogue	a type’s settings over a period
requirement	Core	Built	requirements	a subject must hold a type, over a period
compliance_record	Core	Built	evidence	one completion or result
publish_run	Publishing	Built	publishing	one calculation as of a date
published_requirement_status	Publishing	Built	publishing	a requirement’s status in a run
published_type_score / _sub_group_ / _group_	Publishing	Built	publishing	a score at one level in a run
policy_version	Configuration	Planned	policy	a version of the scoring policy settings
spc_rule_version	Configuration	Planned	spc	SPC parameters for a measure over a period
import_batch	Ingestion	Planned	ingestion	one uploaded source file
import_row	Ingestion	Planned	ingestion	one row of that file and its outcome
source_case	Ingestion (restricted)	Planned	ingestion	a source event: incident, pathway, complication, survey response
measurement	Ingestion	Planned	spc	a monthly or weekly figure for a measure at a site
audit_event	Cross-cutting	Planned	audit	one change made through the API

3. Tables and Attributes

Columns use the database names. **Req.**: Y = NOT NULL. Every table also has a UUID primary key `id` (stable reference that never changes, unlike a name) unless stated.

3.1 subject

Status	Built	**Owner module**	subjects
Replaces	Names tabs, All Staff List	**Grain**	one person / site / organisation

Column	Type	Req.	Value / source	Why we keep it
id	uuid	Y	Generated	Stable key every other table points at; a name change does not break links
kind	enum PERSON / SITE / ORGANISATION	Y	Set at creation	Staff areas (01, 04) track people; Incidents, RTT, Feedback, Complications track sites or the organisation
display_name	text	Y	Form or importer	What screens and reports show
external_ref	text, unique		ESR number, site code	Lets importers match on a source system ID instead of a name
created_at / updated_at	timestamptz	Y	Database	When the subject entered the register

- Leavers are not deleted: their requirements end (requirement.active_to), so history and past scores stay intact.

3.2 subject_alias

Status	Built	**Owner module**	subjects
Replaces	Name text matching in every tool	**Grain**	one name spelling

Column	Type	Req.	Value / source	Why we keep it
subject_id	uuid → subject	Y	Owner subject	Which subject the spelling belongs to
alias	text	Y	As written in the source	Shown to users when resolving imports
normalised	text, unique	Y	Lower-case, trimmed, single-spaced	Fast exact match; uniqueness stops one name pointing at two people
source	text		e.g. ESR, Training Matrix	Explains where an odd spelling came from
created_at	timestamptz	Y	Database	Audit

- Unmatched names in an import become a review task, never a silently created subject.

3.3 compliance_type

Status	Built	**Owner module**	catalogue
Replaces	Compliance Type text in every tool	**Grain**	one compliance type

Column	Type	Req.	Value / source	Why we keep it
name	text	Y	Catalogue admin	Display name
code	text, unique	Y	normalised name	Prevents “DBS” and “ dbs ” becoming two types
created_at	timestamptz	Y	Database	Audit

- Settings are not stored here, because they change over time; they live in versions.

3.4 compliance_type_version

Status	Built	**Owner module**	catalogue
Replaces	Compliance Timeframes tables (12 areas)	**Grain**	one type's settings over a period

Column	Type	Req.	Value / source	Why we keep it
compliance_type_id	uuid → compliance_type	Y	Parent type	Groups the versions
version	int	Y	1, 2, 3…	Human-readable history; unique per type
valid_from / valid_to	date	Y / –	Admin; valid_to set when the next version starts	Point-in-time scoring: a past date uses the settings that applied then
group_level_1, group_level_2	text	Y	Compliance Group – Level 1 / 2	Level 2 selects the compliance rule: Issue = 10, Completion Status or Due Date (DSL §3)
group, sub_group	text	Y	Compliance Group 1 / 2	Primary dashboard hierarchy (Requirements table)
detailed_group, detailed_sub_group	text		Compliance Group 3 / 4	Second hierarchy (Requirements_Detailed); null when the type is not in it (DS-08, DS-10)
core	boolean	Y	Core Requirement	Drives the Core percentages and RAG (decision D-01 in 01)
timeframe_months	int		Compliance Timeframe	Renewal period to calculate due dates; null = never expires (replaces the 10000 sentinel)
threshold_1 … threshold_5	decimal(6,5)	Y	Thresholds 1–5	Band a percentage into RAG scores 1 / 0.8 / 0.6 / 0.4 / 0.2
created_at, created_by	timestamptz, text	Y	API user	Who changed a setting and when

- Database rules: one open version per type; valid_to > valid_from; thresholds 0–1 and not increasing.

3.5 requirement

Status	Built	**Owner module**	requirements
Replaces	Requirements tabs, Active flag, Required (T/F)	**Grain**	a subject must hold a type, over a period

Column	Type	Req.	Value / source	Why we keep it
subject_id	uuid → subject	Y	Form, role template or import	Who must comply
compliance_type_id	uuid → compliance_type	Y	As above	With what
required	boolean	Y	Required (T/F)	Keeps not-required rows visible without counting them
active_from / active_to	date	Y / –	Start; end when someone leaves or the duty stops	Replaces the Active flag with dates, so past dates know who was active
notes	text		Requirements Notes	Context shown next to the status
created_at, created_by, updated_at		Y	API user	Audit

- One open requirement per subject and type (checked in a transaction). Ending a requirement keeps the row.

3.6 compliance_record

Status	Built	**Owner module**	evidence
Replaces	Compliance Data table	**Grain**	one completion or result

Column	Type	Req.	Value / source	Why we keep it
recorded_seq	int, unique, auto	Y	Database sequence	Entry order; breaks ties between records with the same completion date (DS-01)
subject_id	uuid → subject	Y	Resolved from the source	Whose record
compliance_type_id	uuid → compliance_type	Y	Resolved from the source	Which type
completion_date	date		When the check, course or period was completed	Picks the latest record
completion_status	text		e.g. Completed	The Training rule needs it (DSL §3)
due_date	date		Completion + timeframe, or from the source	The due-date rule, expiry flags and weeks non compliant
issue	int		10 = no issue; set by the area's rule or SPC	The rule for Incidents, Feedback, HR Metrics, Complications
notes	text		Compliance Notes	Explains a status to the reader
source	enum MANUAL / IMPORT / MIGRATION	Y	Entry path	Trust and traceability
source_ref	text		Import batch, legacy row or form id (planned: import_batch_id)	Trace a number back to the file or form it came from
recorded_at, recorded_by	timestamptz, text	Y	Database, API user	Point-in-time reads use recorded_at
supersedes_id	uuid → compliance_record, unique		The voided record a correction replaces	Links a correction to what it corrected
voided_at, voided_by, void_reason	timestamptz, text, text		Set once by void or correct	Removes a mistake from scoring without losing it

- A trigger rejects deletes and any update other than setting the void columns once.
- Point-in-time: a run as of D uses records recorded by the end of D and not voided by then.

3.7 publish_run

Status	Built	**Owner module**	publishing
Replaces	Last Updated table, "refresh twice"	**Grain**	one calculation as of a date

Column	Type	Req.	Value / source	Why we keep it
as_of	date	Y	Chosen by the operator or scheduler	Replaces TODAY(); makes the run reproducible
policy_version	text	Y	Policy in force	Which rule settings produced the numbers
status	enum DRAFT / PUBLISHED / DISCARDED	Y	Workflow	Power BI shows only PUBLISHED
warnings	jsonb	Y	Rules engine	Ambiguous latest records, unknown types: a checklist for the reviewer
created_at, created_by		Y	Operator	Who calculated
decided_at, decided_by			Approver	Who published or discarded; must differ from created_by to publish (four eyes)

3.8 published_requirement_status

Status	Built	**Owner module**	publishing
Replaces	Requirements formula columns	**Grain**	a requirement’s status in a run

Column	Type	Req.	Value / source	Why we keep it
run_id	uuid → publish_run	Y	Parent run	Frozen snapshot
requirement_id, subject_id, compliance_type_id	uuid	Y	Requirement	Lets reports join to names and filter
record_id	uuid		Latest record chosen	Drill-down to the evidence behind a status
compliant	boolean	Y	Rules engine	The status
due_date_kind, due_date	text, date	Y / –	DATE / NO_RECORD / NO_EXPIRY_DATE_SET	Explicit states instead of text in a date column (DS-06)
completion_status	text		Latest record	Shown to chasers
compliance, compliance_core	int 1/0		1/0 when required and active	Inputs to percentages; null rows do not count
expiry, weeks_non_compliant	text, int		Rules engine	Chasing lists

3.9 published_type_score, published_sub_group_score, published_group_score

Status	Built	**Owner module**	publishing
Replaces	Requirements / Requirements_Detailed / requirements_Final_Self_Ref roll-ups	**Grain**	one score per level per hierarchy

Column	Type	Req.	Value / source	Why we keep it
run_id	uuid → publish_run	Y	Parent run	Frozen snapshot
hierarchy	enum PRIMARY / DETAILED	Y	Group 1/2 or 3/4	Both dashboard groupings from one run
group, sub_group, compliance_type	text	Y	Catalogue version	Labels at each level (type table has all three)
percent_compliant, percent_compliant_core	float 0–1		Scoring	Gauge values
rag, rag_core	float 0–1		Scoring (threshold bands, Incidents and HR rules)	Gauge colours

- Names are stored as text, not IDs, so a run still reads correctly after the catalogue is renamed.

3.10 policy_version (planned)

Status	Planned	**Owner module**	policy
Replaces	EXCEL_PARITY_POLICY constant in code	**Grain**	one version of the scoring settings

Column	Type	Req.	Value / source	Why we keep it
version	text, unique	Y	e.g. excel-parity-1, 2027-01	Referenced by publish_run.policy_version
settings	jsonb	Y	CompliancePolicy object	Client decisions (DS-02, DS-03, DS-04, DS-07, DS-13) become data, not code
valid_from	date	Y	Admin	Which version applies to a run date
created_by, approved_by, created_at		Y	Admin + approver	A rule change is approved like a publish
notes	text		Decision reference	Links to the client decision that caused it

3.11 spc_rule_version (planned)

Status	Planned	**Owner module**	spc
Replaces	SPC sheets in Incident Management and Survey tools	**Grain**	SPC parameters for a measure

Column	Type	Req.	Value / source	Why we keep it
compliance_type_id	uuid → compliance_type	Y	The SPC measure (e.g. Falls SPC)	Which measure the rules apply to
version, valid_from, valid_to		Y	Admin	Versioned like the catalogue
method	text	Y	e.g. XmR / p-chart	How limits are calculated (open in specs 05, 07)
baseline_from, baseline_to	date		Baseline period	Limits come from a fixed baseline, not a moving window
parameters	jsonb	Y	Sigma, run length, trend length	The signals that set Issue below 10

3.12 import_batch (planned)

Status	Planned	**Owner module**	ingestion
Replaces	Paste of an export into a tool	**Grain**	one uploaded file

Column	Type	Req.	Value / source	Why we keep it
source	enum DATIX / SYSTMONE_RTT / SURVICATE / ESR / WFS / CIC_LD / COMPLICATIONS / LEGACY_XLSM / MANUAL_CSV	Y	Uploader or scheduler	Selects the adapter and its header contract
file_name, file_sha256	text	Y	Upload	sha256 unique per source: the same file loaded twice is a no-op
period_from, period_to	date		From the file or uploader	Detects gaps and overlaps between monthly or weekly loads
contract_version	text	Y	Adapter	Which column layout was expected
status	enum RECEIVED / VALIDATED / REJECTED / PROMOTED	Y	Workflow	Nothing reaches evidence until a valid batch is promoted
row_count, accepted_count, rejected_count	int	Y	Adapter	Reconciliation shown to the operator
errors	jsonb		Adapter	File-level problems (missing column, wrong period)
uploaded_by/at, promoted_by/at			Users	Audit

3.13 import_row (planned)

Status	Planned	**Owner module**	ingestion
Replaces	Rows of the pasted export	**Grain**	one source row

Column	Type	Req.	Value / source	Why we keep it
batch_id	uuid → import_batch	Y	Parent	Reconciliation
row_number	int	Y	File	Point users at the exact row to fix
raw	jsonb (restricted)	Y	File row	Trace any number back to the source; kept for a limited period
status, errors	enum, jsonb	Y / –	VALID / INVALID / DUPLICATE / PROMOTED	Row-level validation results
dedupe_key	text	Y	Adapter (e.g. ESR id + course + date)	Duplicate detection across batches
subject_id, compliance_type_id, record_id	uuid		Resolution and promotion	Which subject it matched and which record it created

3.14 source_case (planned, restricted)

Status	Planned	**Owner module**	ingestion
Replaces	Datix rows, RTT pathways, complication rows, survey responses in tools	**Grain**	one source event

Column	Type	Req.	Value / source	Why we keep it
source, external_id	enum, text	Y	Datix ID, pathway ID…	Unique per source; an updated Datix case updates its row instead of duplicating
subject_id	uuid → subject	Y	Site / organisation resolved	Who the event counts against
event_type	text	Y	Incident category, survey name…	Feeds the area rules
occurred_on, reported_on, closed_on	date		Source dates	Timeliness rules (e.g. reported within 48 h, review panel in time)
harm_level	text		Datix	Moderate Harm and SPC measures
attributes	jsonb		Remaining source fields	Area-specific fields without a table per source
contains_patient_data	boolean	Y	Adapter	Drives access control and retention
import_batch_id	uuid → import_batch	Y	Loader	Lineage

- Stored in a restricted schema; never exposed through reporting views. Needs IG approval before go-live (D-08).

3.15 measurement (planned)

Status	Planned	**Owner module**	spc
Replaces	Monthly figures on SPC sheets	**Grain**	one figure for a measure, subject and period

Column	Type	Req.	Value / source	Why we keep it
subject_id, compliance_type_id	uuid	Y	Site + measure	What is measured where
period_start, period_end	date	Y	Calendar month or week	SPC needs an ordered series
value, denominator	numeric	Y / –	Derived from source_case or the source file	Counts and rates
import_batch_id	uuid	Y	Loader	Lineage; a reload supersedes the earlier figure

3.16 audit_event (planned)

Status	Planned	**Owner module**	audit
Replaces	Nothing (no audit today)	**Grain**	one change

Column	Type	Req.	Value / source	Why we keep it
occurred_at, actor, request_id		Y	API	Who, when, which request
action, entity, entity_id	text	Y	API	What changed
before, after	jsonb		Service	Changes to tables that are not append-only (subjects, requirements, catalogue)

4. Relationships

From	To	Card.	Key / delete rule	Why
subject_alias	subject	N:1	subject_id, RESTRICT	A subject has many spellings; an alias belongs to one subject
requirement	subject	N:1	subject_id, RESTRICT	A person has many requirements; deleting a person must not erase obligations
requirement	compliance_type	N:1	compliance_type_id, RESTRICT	Many subjects need the same type
compliance_type_version	compliance_type	N:1	compliance_type_id, RESTRICT	Settings change over time; one open version
compliance_record	subject	N:1	subject_id, RESTRICT	History per person or site
compliance_record	compliance_type	N:1	compliance_type_id, RESTRICT	History per type
compliance_record	compliance_record	1:0..1	supersedes_id, unique	A correction replaces exactly one voided record
requirement ↔ compliance_record	(no FK)	matched	subject + type at the as-of date	A record can satisfy a requirement that started later, and survives requirement changes (as in the workbook)
publish_run	policy_version	N:1	policy_version (text; FK when the table exists)	Every run names the rules that produced it
published_* tables	publish_run	N:1	run_id, CASCADE	Snapshot rows belong to their run
published_requirement_status	requirement, record	N:1	ids stored, no FK	Snapshots stay valid even if source rows are later voided
import_row	import_batch	N:1	batch_id	Rows belong to one file
compliance_record	import_batch	N:0..1	import_batch_id (planned)	Lineage for imported records; manual records have none
source_case	subject, import_batch	N:1	subject_id, import_batch_id	Events belong to a site and arrive in a file
measurement	subject, compliance_type	N:1	ids	A series per site and measure
spc_rule_version	compliance_type	N:1	compliance_type_id	Each SPC measure has versioned parameters

RESTRICT on core tables means nothing that has history can be deleted; ending or voiding is the only way to retire it.

5. Processing

5.1 Pipeline

Stage	What happens	Module	On failure
1 Ingest	File uploaded or fetched; sha256 checked; batch RECEIVED	ingestion	Same file → returns the existing batch
2 Validate	Header contract, types, dates, period; rows VALID / INVALID	ingestion	Batch REJECTED with row errors; nothing written
3 Resolve	Names → subjects via aliases; type names → catalogue	ingestion + subjects	Unmatched names go to a review list
4 Derive	Area rules turn rows into record values (due dates, Issue); SPC for measures	ingestion + spc	Rows without enough data flagged
5 Promote	One transaction appends compliance_record rows; batch PROMOTED	evidence	Rolls back as a whole
6 Calculate	Rules + scoring as of a date → DRAFT run with warnings	publishing	No change to published data
7 Review	Approver compares with the last published run, checks warnings	publishing + UI	Discard and fix data
8 Publish	Second person approves; run PUBLISHED	publishing	Four-eyes rule enforced in DB
9 Report	Power BI refreshes from reporting_* views	views	Previous run stays visible

5.2 Processing per area (from the recordings)

Area	Source / frequency	Derive step	Record produced
01 Staff	Forms (API), as needed	Due = completion + timeframe (rule R-01 to confirm: due date or day before)	One record per completion, subject = person
04 Training	ESR, WFS, L&D reports; monthly	Latest completion per person and course; dedupe on person + course + date; due from course timeframe	New completions only (no delete-and-replace); Completion Status = Completed
06 RTT	SystmOne breach report; weekly	Breaches per service and week → measurement; rule sets Issue	Weekly record per site; group to confirm (DS-11)
02/07 Incidents	Datix export; monthly	source_case per incident; timeliness checks; monthly counts per harm level → measurement; SPC → Issue	Monthly record per site and measure (e.g. Moderate Harm, Falls SPC)
05 Feedback	Survicate (4 surveys); monthly	Responses → measures (scores, counts) → SPC → Issue	Monthly record per site and measure
03 Complications	SystmOne + appointments; monthly	Patient-level source_case (restricted) → rate per service → Issue	Monthly record per service; no patient data leaves the restricted area
Legacy	Compliance Data.xlsm + 11 tools; once	Rows mapped to subjects and types; Unique Lookups dropped	MIGRATION records with source_ref = sheet + row

5.3 Monthly cycle

When	Who	Step
Working day 1–3	Scheduler / operator	Imports for the previous month; fix rejected rows and unmatched names
Day 4	Operator	Calculate draft as of month end; review warnings and changes against last month
Day 5	Approver (different person)	Approve and publish; Power BI refresh picks it up
Any time	Operator	Manual records and corrections; they appear in the next run
Weekly	Scheduler	RTT import; expiring-items list for chasing

5.4 Rules that keep processing safe

- **Idempotent:** a file hash or an Idempotency-Key header makes retries safe.
- **Short transactions:** one per promote and one per calculate; no external calls inside them.
- **Point-in-time:** recalculating a past date uses the records and settings as they stood then.
- **Corrections never edit:** void + supersede in one transaction; the next run reflects it.
- **Nothing reaches Power BI without approval:** drafts are invisible to the views.

6. Final Data Store and Column Lineage

Power BI reads only these views. Each column below says where its value comes from.

6.1 reporting_requirement_status (one row per requirement in the latest published run)

Column	Comes from	Excel equivalent
as_of	publish_run.as_of	TODAY() when the file was opened
run_id, requirement_id	publish_run, requirement	—
subject_id, subject_name	subject.id, subject.display_name	Name
compliance_type_id, compliance_type_name	compliance_type	Compliance Type
record_id	Latest record for subject + type (rules engine)	Most Recent Record / Unique Lookup 4
compliant	Rule chosen by the version's Level 2 group: Issue = 10; Completion Status filled; or due date after as_of	Compliant (row 192)
due_date_kind, due_date	Latest record's due_date, or NO_RECORD / NO_EXPIRY_DATE_SET	Due Date (row 212)
completion_status	Latest record	Compliance Completion Status (row 215)
compliance, compliance_core	1/0 if requirement.required and active at as_of (and version.core)	Compliance, Compliance (Core) (216–217)
expiry	due_date before as_of + 3 months (10 working days for Ongoing Audit)	Expires within 3 months (193)
weeks_non_compliant	round((as_of − due_date) / 7) when non compliant	Weeks Non Compliant (194)

6.2 reporting_type_score / reporting_sub_group_score / reporting_group_score

Column	Comes from	Excel equivalent
hierarchy	PRIMARY: version.group / sub_group; DETAILED: detailed_group / detailed_sub_group	Requirements vs Requirements_Detailed
group, sub_group, compliance_type	Catalogue version valid at as_of	Compliance Groups / Sub Groups / Type
percent_compliant (type)	Average of compliance over the type's requirements	Compliance Type – % Compliant (218)
rag (type)	Band of the % against the version's Thresholds 1–5	Compliance Type – RAG Score (220)
percent_compliant (sub group)	Average of type percentages (Onboarding excluded)	Sub Groups – % Compliant (234)
rag (sub group)	Incidents rule, HR Metrics SPC rule, or zero-type gate then band	Sub Groups – RAG Score (243)
percent_compliant, rag (group)	Average of sub-group %; band against row-weighted average thresholds	Groups – % Compliant / RAG (236, 245)
*_core	Same, using core requirements only	(Core) columns
as_of	publish_run.as_of	Last Reviewed Date (approximately)

6.3 Where compliance_record values come from, by source

Column	Staff (form)	Training	RTT	Incidents	Feedback	Complications	Legacy
subject_id	Picked person	ESR id / name alias	Site	Datix site	Survey site	Service	Name alias
compliance_type_id	Picked type	Course → type map	RTT measure	Measure	Measure	Measure	Type name
completion_date	Typed	Course date	Week end	Month end	Month end	Month end	Column
completion_status	Typed	Completed	—	—	—	—	Column
due_date	+ timeframe	+ timeframe	Next period	Next period	Next period	Next period	Column
issue	—	—	Rule	Rule / SPC	SPC	Rule	Column
source	MANUAL	IMPORT	IMPORT	IMPORT	IMPORT	IMPORT	MIGRATION

"Next period" and the Issue rules for RTT, Incidents, Feedback and Complications are listed for client confirmation in each folder's pre-build PDF.

7. API

REST + JSON under `/api/compliance/v1`, served by the same NestJS app. Each endpoint calls one module's service; controllers only validate input, check the role and map errors.

7.1 Conventions

Topic	Rule	Why
Auth	Bearer token from the appointment system's identity provider; NestJS guard checks role	One login for both systems
Roles	Viewer, Operator, Approver, Admin, Integration (service account)	Four eyes needs separate Operator and Approver
Validation	DTOs with class-validator; dates as YYYY-MM-DD	Reject bad data at the edge
Errors	Problem Details (RFC 9457): Validation 400, Forbidden 403, NotFound 404, Conflict 409	Errors already exist as plain classes in the code
Idempotency	Idempotency-Key header on POST records and imports	Safe retries from importers and flaky networks
Lists	Cursor pagination, filters by subject, type, as-of	Bounded responses
Audit	Every write logs actor and request id to audit_event	IG and "who changed this?"
Versioning	/v1 in the path; additive changes only	Power BI and other clients keep working

7.2 Endpoints

Method + path	Role	Does	Why needed
GET /subjects?query=	Viewer	Search subjects by name or alias	Pick a person or site
POST /subjects	Operator	Create a subject with aliases	New starter or site
POST /subjects/{id}/aliases	Operator	Add a spelling	Resolve import mismatches
GET /subjects/resolve?name=	Integration	Name → subject	Importers and forms
GET /compliance-types?asOf=	Viewer	Catalogue as of a date	Forms and review
POST /compliance-types	Admin	New type with version 1	New obligation
POST /compliance-types/{id}/versions	Admin	New settings from a date	Threshold or core change
GET /requirements?subjectId=&asOf=	Viewer	Requirements of a subject	Profile screen
POST /requirements, POST /requirements/bulk	Operator	Assign one, or a role template	Replaces picking 22 types by hand (01)
POST /requirements/{id}/end	Operator	End a requirement	Leavers; keeps history
GET /records?subjectId=&typeId=	Viewer	History incl. voided	Evidence trail
POST /records	Operator	Add a completion	Manual entry (01)
POST /records/{id}/corrections	Operator	Void + replace with reason	Fix mistakes without overwriting
POST /records/{id}/void	Operator	Void with reason	Remove duplicates
POST /imports (multipart, source)	Operator / Integration	Upload and validate a file	Replaces copy-paste
GET /imports/{id}, GET /imports/{id}/rows?status=	Operator	Batch summary and row errors	Fix and reconcile
POST /imports/{id}/promote | /reject	Operator	Write records or abandon	Explicit control
POST /publish-runs {asOf}	Operator	Calculate a draft	Monthly cycle
GET /publish-runs/{id}, /{id}/diff?against=latest	Operator / Approver	Draft, warnings and changes	Informed approval
POST /publish-runs/{id}/approve | /discard	Approver	Publish (not by its creator) or discard	Four eyes
GET /publish-runs/latest	Viewer	Latest published run	Status banner
GET /reports/requirement-status, /reports/expiring?within=	Viewer	Status and chasing lists	In-app lists without Power BI
GET /policies, POST /policies	Viewer / Admin	Scoring policy versions	Record client decisions
GET /audit?entity=&id=	Admin	Change history	IG queries
GET /health/live, /health/ready	None	Liveness and DB readiness	Hosting and monitoring

8. Build Guide

How I would deliver the whole system, in order, with what “done” means at each step.

8.1 Phases

#	Phase	Deliverables	Done when
0	Foundations (done)	Rules + scoring; storage modules; migrations; reporting views; CI	Merged with unit and integration tests (repo PR #1)
1	API + auth	Controllers, DTOs, guards and roles, error mapping, audit_event, health checks, OpenAPI	Staff Management (01) usable end to end in a test environment
2	Legacy migration + parity	LEGACY_XLSM importer; policy_version table; parity report	Every Compliant / % / RAG matches Excel or is an agreed DS-finding difference
3	Importers, area by area	Training → RTT → Incidents → Feedback → Complications; import_batch, import_row, source_case	Each area runs one month in parallel with its Excel tool, then the tool is retired
4	SPC	measurement, spc_rule_version, SPC engine (pure, tested)	SPC signals match the tools’ SPC sheets for 12 months of history
5	Scheduler + chasing	Scheduled imports and drafts; expiring lists and emails	Monthly cycle runs with only review and approval by people
6	Remaining areas	Estates, organisation, policy, medical devices, audit, sickness/leave	Configured from walkthroughs; all 12 areas on the platform

8.2 Non-functional requirements

Area	Requirement
Security	Role-based access; least privilege DB users (app user cannot ALTER or bypass triggers); secrets in the host’s vault
Information governance	DPIA before patient data; restricted schema for source_case and import_row.raw; no patient fields in views
Retention	Compliance records 7 years (template tab 02); raw import rows shorter, set by IG
Backup and recovery	Daily backups with point-in-time recovery; restore rehearsed before go-live
Observability	Structured logs with request id; metrics for import and publish duration; alert on failed scheduled imports
Performance	About 4,000 records growing by up to 20 a week: a full calculation takes seconds; no queue or cache needed
Change control	Migrations reviewed in PRs; policy and catalogue changes versioned and approved

8.3 Testing strategy

Layer	Test	Exists
Rules and scoring	Unit tests for every rule and DS finding	Yes (52)
Storage	Integration tests against PostgreSQL: constraints, trigger, point-in-time, four eyes	Yes (17)
Architecture	dependency-cruiser: pure core, independent modules	Yes
API	e2e tests per endpoint with roles	Phase 1
Importers	Golden files per source (anonymised samples)	Phase 3
Parity	Excel vs new scores on the real Compliance Data.xlsm	Phase 2

8.4 Risks and mitigations

Risk	Mitigation
Client decisions arrive late	Policy settings default to Excel behaviour; decisions become new policy versions
Source exports change columns	Header contracts per source; a changed file is rejected, not mis-loaded
Name matching errors	Aliases plus external refs; unmatched names reviewed, never auto-created
Parity gaps erode trust	Parity report per area before retiring each tool
Patient data exposure	Restricted schema, IG sign-off, no patient fields in reporting
One person’s knowledge again	Specs, ADRs, tests and policy versions in the repo

8.5 What I need from you and the client

Item	For phase	Reference
Appointment data store’s auth method and roles (or repo access)	1	—
Copy of Compliance Data.xlsm and the 11 tools, with queries and VBA	2	PreBuild F-01, F-02
Anonymised sample export per source	3	PreBuild F-xx per folder

Item	For phase	Reference
Rule decisions: blank due date, Training lapse, RTT group, Moderate Harm, core flags	2–3	DS-02, DS-03, DS-07, DS-11, D-01
SPC method and parameters	4	Specs 05, 07
IG approval (DPIA) for patient-level data	3 (Incidents, Complications)	D-08